

UNIVERSIDAD TECNOLOGICA DE AGUASCALIENTES

Licenciatura en Ingenierías de la Información e Innovación Digital

ITIID

Fundamentos de Inteligencia Artificial

Parcial 2
Proyecto Final:
Sistema inteligente de planificación, optimización y toma de decisiones adversarias
Presenta:

Aaron Huerta Rodriguez 240284

Docente:
Froylan Alonso Perez Alanis

Índice

1. Resumen ejecutivo ... 3
2. Planteamiento del problema y supuestos ... 4
2.1 Descripción del problema ... 4
2.2 Escenario ... 4
2.3 Supuestos ... 4
3. Arquitectura e integración ... 5
3.1 Visión general ... 5
3.2 Componentes principales ... 6
3.3 Decisiones de integración ... 6
4. Fundamento y diseño de cada algoritmo ... 7
4.1 Búsqueda heurística: Greedy Best-First y A* ... 7
4.2 Búsqueda local: Hill Climbing ... 7
4.3 Recocido Simulado ... 7
4.4 Algoritmo Genético ... 7
4.5 Teoría de juegos: juego simultáneo 3 × 3 ... 8
4.6 Búsqueda adversaria: Minimax y poda Alfa-Beta ... 8
4.7 Complejidad ... 8
5. Diseño experimental y parámetros ... 9
5.1 Experimentos ... 9
5.2 Parámetros ... 9
5.3 Métricas y procedimiento ... 10
6. Resultados cuantitativos y visualizaciones ... 10
6.1 Búsqueda heurística (E1) ... 10
6.2 Matriz de costos y ruta óptima ... 10
6.3 Hill Climbing (E2) ... 11
6.4 Recocido Simulado vs Algoritmo Genético (E3) ... 12
6.5 Teoría de juegos (E4) ... 13
6.6 Minimax vs Alfa-Beta (E5) ... 13
6.7 Caso límite (E6) ... 13
7. Análisis comparativo y discusión ... 14
7.1 A* frente a Greedy ... 14
7.2 Hill Climbing, Recocido Simulado y Algoritmo Genético ... 14
7.3 Decisión estratégica ... 14
8. Pruebas, limitaciones y amenazas a la validez ... 15
8.1 Pruebas realizadas ... 15

8.2 Limitaciones y amenazas a la validez 15
9. Conclusiones y trabajo futuro 15
9.1 Conclusiones 15
9.2 Trabajo futuro 16
10. Referencias y anexos técnicos 16
10.1 Referencias 16
Anexo A. Archivos y reproducibilidad 18
Anexo B. Detalle de Hill Climbing (10 ejecuciones) 18
Anexo C. Preguntas guía para la defensa individual 18

1. Resumen ejecutivo

Este informe documenta el diseño, la implementación y la evaluación de un sistema de apoyo a decisiones logísticas. El problema consiste en planear la ruta de un vehículo que parte de un origen **S**, visita tres puntos de entrega **D1, D2 y D3** y termina en un destino **F**, sobre una cuadrícula de 12 × 12 con terreno de costo variable (1, 3 o 5) y obstáculos infranqueables. Sobre esa misma base se estudia además una situación de competencia: la elección de ruta frente a un competidor que puede bloquear (juego simultáneo) y una negociación por turnos por recursos limitados (juego de suma cero).

La solución integra seis componentes: (1) búsqueda heurística con **Greedy Best-First** y **A*** con distancia de Manhattan; (2) **Hill Climbing** para ordenar las entregas usando una matriz de costos precalculada con A*; (3) **Recocido Simulado** y **Algoritmo Genético** comparados en 20 ejecuciones; (4) análisis de un juego de forma normal 3 × 3 con equilibrio de Nash; (5) **Minimax** y **poda Alfa-Beta** sobre una negociación de recursos; y (6) una interfaz en **Streamlit** que reproduce el notebook y presenta sus salidas.

Resultados principales:

- La ruta de menor costo es **S → D1 → D3 → D2 → F con costo total 36**; coincide con la enumeración exhaustiva de las 6 permutaciones posibles.
- A* encontró siempre rutas óptimas; Greedy fue más rápido pero produjo rutas peores en 2 de 3 trayectos (costo 22 frente a 14 y 15 frente a 11), a cambio de expandir menos nodos.
- Recocido Simulado obtuvo costo 36 en las 20 ejecuciones (desviación 0) con 50 evaluaciones; el Algoritmo Genético obtuvo 36 en 19 de 20 (una ejecución terminó en 38) con 200 evaluaciones y un tiempo ≈ 9.5 veces mayor.
- El juego 3 × 3 tiene un único equilibrio de Nash puro: **(Ruta Segura, No Bloquear)** con pagos (5, 4); no hay estrategias estrictamente dominantes.
- Alfa-Beta calculó el mismo valor que Minimax (11) explorando **246 nodos en lugar de 3,620** (reducción de 93.20 %).

Alcance de las conclusiones. Con solo tres entregas el espacio de búsqueda contiene 6 soluciones, por lo que cualquier método razonable alcanza el óptimo. Las diferencias entre métodos se interpretan por su eficiencia y estabilidad, no por su capacidad para resolver un problema difícil; la sección 8 detalla esta y otras limitaciones.

2. Planteamiento del problema y supuestos

2.1 Descripción del problema

Una empresa de distribución opera una flota desde un punto de origen y debe atender tres entregas antes de llegar a un destino final. La red se modela como una cuadrícula de 12 × 12 celdas. Cada celda tiene un costo de entrada que representa tiempo o combustible: **1** (terreno llano), **3** (congestión) o **5** (zona difícil); algunas celdas son obstáculos (**X**) y no pueden transitarse. Se busca minimizar el costo total del recorrido y, de forma complementaria, analizar decisiones estratégicas frente a un competidor.

Se plantean dos subproblemas encadenados. El primero es de **ruteo**: encontrar el camino de mínimo costo entre dos puntos (búsqueda en grafo) y decidir en qué orden visitar las entregas (optimización combinatoria). El segundo es de **decisión estratégica**: elegir una estrategia cuando el resultado depende también de las acciones de un rival, tanto en una interacción simultánea como en una secuencial.

2.2 Escenario

Tabla 1. Configuración del escenario (archivo mapa_logistico.csv).

Elemento	Posición (fila, col.)	Descripción
Origen **S**	(0, 0)	Salida de la flota
Destino **F**	(11, 11)	Llegada final
Entrega **D1**	(1, 10)	Esquina superior derecha
Entrega **D2**	(9, 4)	Zona inferior central
Entrega **D3**	(5, 11)	Borde derecho
Zona de congestión (costo 3)	filas 3–5, cols. 3–5	Bloque de 3 × 3 celdas
Zona difícil (costo 5)	filas 8–10, cols. 8–10	Bloque de 3 × 3 celdas (con (8,8) como obstáculo)
Obstáculos (8 celdas)	(2,2), (2,3), (2,4), (6,8), (7,8), (8,8), (10,1), (10,2)	Celdas infranqueables

2.3 Supuestos

- **Movimiento:** cuatro direcciones (arriba, abajo, izquierda, derecha); no hay movimientos diagonales.
- **Costo de un paso:** es el costo de la celda a la que se entra. Las celdas S, F y D1–D3 cuestan 1.
- **Costo mínimo unitario:** todas las celdas transitables cuestan al menos 1, lo que hace admisible y consistente la distancia de Manhattan.

- **Un solo vehículo** que visita cada entrega exactamente una vez, en cualquier orden, con recorrido S → (D1, D2, D3) → F. No se consideran capacidad, ventanas de tiempo ni tráfico dinámico.
- **Entorno estático y determinista:** el mapa no cambia durante la ejecución y los costos son conocidos.
- **Costo entre puntos clave:** se usa el costo óptimo calculado con A*, que es simétrico en este mapa.
- **Juego simultáneo:** información completa; los pagos son valores ordinales de utilidad definidos por el diseñador, no derivados del mapa.
- **Negociación por turnos:** los recursos tienen valores conocidos, los jugadores alternan elecciones, el juego es de suma cero y la búsqueda se limita a profundidad 5.

3. Arquitectura e integración

3.1 Visión general

El sistema se organiza en fases que comparten estado dentro de un único notebook (`proyecto2.ipynb`). La Fase 1 construye el entorno a partir de un CSV; la Fase 2 implementa las búsquedas sobre ese entorno; las Fases 3 y 4 consumen una **matriz de costos mínimos** generada con A* para evaluar permutaciones de entregas sin volver a buscar en el mapa; las Fases 5 y 6 son módulos independientes de decisión estratégica; y las Fases 7 y 8 consolidan, grafican y exportan los resultados de la Fase 4. Finalmente, `app.py` ejecuta el notebook y presenta sus salidas.

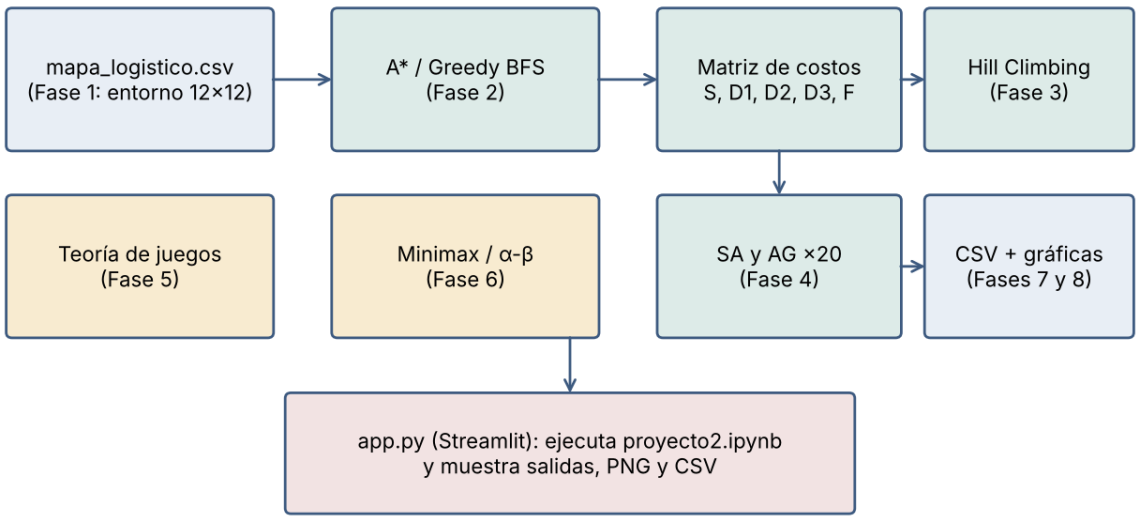


Figura 1. Arquitectura e integración de las fases del sistema.

3.2 Componentes principales

Tabla 2. Componentes del sistema.

Componente	Elemento de código	Responsabilidad
Entorno	`EntornoLogistico`, `generar_csv_escenario()`	Genera y lee el CSV, identifica S, F, D1–D3 y obstáculos; valida celdas y devuelve costos.
Búsqueda	`greedy_bfs`, `a_star`, `heuristica_manhattan`	Calculan ruta, costo, nodos expandidos y tiempo entre dos celdas.
Matriz de costos	`matriz_costos`, `calcular_costo_ruta`	Precalcula con A* el costo entre los 5 puntos clave y evalúa un orden de entregas en O(n).
Búsqueda local	`hill_climbing`	Mejora una permutación con vecindad por intercambio (swap).
Metaheurísticas	`recocido_simulado`, `algoritmo_genetico`	Optimizan el orden de entregas; se ejecutan 20 veces cada una.
Teoría de juegos	`matriz_pagos`, mejores respuestas, Nash	Analiza el juego simultáneo 3 × 3.
Adversaria	`NegociacionLogistica` (`minimax`, `alfabeta`)	Compara ambos algoritmos y cuenta nodos y podas.
Salidas	Fases 7 y 8	Genera `graficas_resultados.png` y `datos_experimentales.csv`; prueba un caso límite.
Interfaz	`app.py` (Streamlit)	Ejecuta las celdas con `exec` en un espacio de nombres común, captura la salida y muestra código, texto, imagen y tabla.

3.3 Decisiones de integración

- **Precomputación con A*.** Evaluar una permutación solo requiere sumar 4 valores de la matriz; así, Hill Climbing, Recocido Simulado y el Algoritmo Genético no ejecutan búsquedas en el mapa durante la optimización.
- **Reproducibilidad.** Se fija la semilla `SEED = 42` para `random` y `numpy`, y el escenario se materializa en un CSV versionable.
- **Interfaz.** `app.py` usa el backend `Agg` de matplotlib para guardar las gráficas como PNG, redirige `stdout` para capturar los `print`, almacena el resultado con `st.cache_data` (invalidado por la fecha de modificación del notebook) y detecta artefactos por nombre de archivo.

Observación sobre el notebook. La primera celda contiene un prototipo previo (escenario aleatorio `escenario.csv`, puntos I, E1–E3, una sola comparación I → E1). La segunda celda redefine `entorno` y `heuristica_manhattan` con el escenario definitivo (`mapa_logistico.csv`), por lo que el resto del sistema usa únicamente este último. Como todas las celdas comparten variables globales, las fases dependen del orden de ejecución.

4. Fundamento y diseño de cada algoritmo

4.1 Búsqueda heurística: Greedy Best-First y A*

Ambos algoritmos mantienen una frontera en una cola de prioridad (`heapq`) y expanden primero el nodo de menor valor de evaluación. Difieren en la función de prioridad:

$$\textit{Greedy: } f(n) = h(n) \qquad A^*\textit{: } f(n) = g(n) + h(n)$$

donde g(n) es el costo real acumulado desde S y h(n) la distancia de Manhattan al objetivo: |x_n − x_obj| + |y_n − y_obj|.

- **Admisibilidad y consistencia.** Como cada paso cuesta al menos 1, la distancia de Manhattan nunca sobreestima el costo restante (admisible). Además, entre celdas vecinas h cambia en a lo sumo 1, que es menor o igual al costo del paso (consistente). Por ello A* devuelve rutas óptimas.
- **Greedy** ignora g(n): suele expandir menos nodos, pero no garantiza optimalidad, como se observa en los resultados.
- **Detalles de implementación.** A* actualiza un nodo solo si encuentra un costo g menor (`costo_g_minimo`); Greedy mantiene un conjunto de visitados. Ambos guardan la ruta completa en cada elemento de la frontera, lo que simplifica el código pero copia listas en cada inserción.
- **Sin solución.** Si el objetivo no es alcanzable, ambos devuelven `None` al agotar la frontera.

4.2 Búsqueda local: Hill Climbing

El espacio de búsqueda son las permutaciones de {D1, D2, D3}; el costo de una solución es S→primera + entre entregas + última→F, leído de la matriz de costos. La **vecindad** de una permutación son todas las que resultan de intercambiar dos posiciones (3 vecinos para n = 3). El algoritmo es de *mejor mejora* (steepest ascent): en cada iteración evalúa toda la vecindad, se mueve al mejor vecino si mejora estrictamente el costo actual y se detiene cuando ninguno mejora (óptimo local o meseta). Se ejecuta desde 10 soluciones iniciales aleatorias.

4.3 Recocido Simulado

Parte de una permutación aleatoria y en cada paso genera un vecino por intercambio aleatorio. Si el vecino mejora (Δ < 0) se acepta; si empeora, se acepta con probabilidad de Metropolis:

$$P(\Delta, T) = \exp(-\Delta / T)$$

La temperatura se enfría geométricamente, T ← α·T. Parámetros usados: T_0 = 100, α = 0.9, T mínima = 0.1 y un tope de 50 evaluaciones. Con estos valores, T llega a ≈ 0.57 tras 49 enfriamientos, por lo que el criterio de paro efectivo es el tope de evaluaciones y no la temperatura. Se conserva la mejor solución encontrada.

4.4 Algoritmo Genético

- **Representación:** permutación de las entregas (sin duplicados ni faltantes).

- **Población y generaciones:** 10 individuos, 20 generaciones (200 evaluaciones).
- **Selección:** torneo con k = 2.
- **Cruza:** variante simplificada de orden (OX1). Con solo 3 genes se hereda un gen del padre 1 en una posición aleatoria y el resto se completa con los genes faltantes en el orden del padre 2; siempre produce una permutación válida.
- **Mutación:** intercambio de dos genes con probabilidad 0.2.
- **Elitismo:** el mejor individuo global se copia a la siguiente generación.

4.5 Teoría de juegos: juego simultáneo 3 × 3

El Agente Logístico (J1, filas) elige entre Ruta Segura, Ruta Principal y Ruta Secundaria; el Competidor (J2, columnas) elige entre Bloquear Principal, Bloquear Secundaria y No Bloquear. Se define una matriz de pagos (u_1, u_2) y se calculan: (a) las **mejores respuestas** de cada jugador a cada acción del otro, (b) las estrategias **estrictamente dominantes** y (c) los **equilibrios de Nash puros**, como la intersección de ambos conjuntos de mejores respuestas, es decir, celdas donde ningún jugador gana desviándose unilateralmente.

4.6 Búsqueda adversaria: Minimax y poda Alfa-Beta

Dos jugadores alternan la elección de un recurso de la lista [15, 10, 8, 5, 3, 2, 1]; MAX es el agente y MIN el competidor. La utilidad es la diferencia de puntos acumulados (score_MAX − score_MIN) y la búsqueda se corta a profundidad 5 (el juego completo tendría 7 niveles). **Minimax** recorre todo el árbol. **Alfa-Beta** conserva los límites α (mejor valor garantizado para MAX) y β (para MIN) y deja de explorar los hermanos restantes de un nodo cuando se cumple

$$\alpha \geq \beta$$

lo que no altera el valor de la raíz. El orden de los recursos (de mayor a menor) actúa como un ordenamiento de movimientos favorable para la poda.

4.7 Complejidad

Sean V = 144 celdas, E ≈ 4V aristas, n = 3 entregas, b el factor de ramificación y d la profundidad.

Tabla 3. Complejidad teórica y valores observados.

Algoritmo	Tiempo (peor caso)	Comentario en este proyecto
Greedy BFS	O(b^m) en general; O(E log V) con visitados	No óptimo. 12 a 16 expansiones en los trayectos probados.
A*	O(b^d) en general; acotado por reexpansiones y O(E log V) por inserción en el heap	Óptimo con h admisible. 22 a 48 expansiones en los trayectos probados. Copiar la ruta en cada inserción añade un factor O(L).
Matriz de costos	20 ejecuciones de A* (pares ordenados de 5 puntos)	Por simetría bastarían 10.
Hill Climbing	O(n²) vecinos × O(n) evaluación por iteración	n! = 6 estados; cada iteración evalúa 3 vecinos.

Algoritmo	Tiempo (peor caso)	Comentario en este proyecto
Recocido Simulado	O(I · n), con I = 50	50 evaluaciones frente a 6 soluciones posibles.
Algoritmo Genético	O(G · P · n) = 20 · 10 · 3	200 evaluaciones; además costo de selección, cruza y ordenamiento.
Minimax	O(b^d); aquí 7·6·5·4·3 = 2,520 hojas	3,620 nodos en total (1 + 7 + 42 + 210 + 840 + 2,520).
Alfa-Beta	O(b^d) peor caso; O(b^(d/2)) con ordenamiento ideal	246 nodos observados, 68 podas.
Mejores respuestas / Nash	O(m · n) sobre la matriz de pagos	Matriz 3 × 3.

5. Diseño experimental y parámetros

5.1 Experimentos

Tabla 4. Experimentos realizados.

ID	Experimento	Descripción y métricas
E1	Greedy vs A*	Tres trayectos: S→D1, D1→D2 y D2→D3. Métricas: costo de la ruta, nodos expandidos y tiempo. (Más una comparación previa I→E1 en el escenario prototipo.)
E2	Hill Climbing	10 soluciones iniciales aleatorias. Métrica: costo inicial y costo del óptimo local alcanzado.
E3	Recocido Simulado vs Algoritmo Genético	20 ejecuciones independientes por método. Métricas: mejor costo, media, desviación estándar, evaluaciones promedio y tiempo promedio.
E4	Juego simultáneo	Mejores respuestas, dominancia y equilibrio de Nash.
E5	Minimax vs Alfa-Beta	Valor óptimo, nodos explorados, podas y porcentaje de reducción.
E6	Caso límite	Destino (2,2), que es un obstáculo: verificar manejo de entrada inválida y ausencia de solución.

5.2 Parámetros

Tabla 5. Parámetros de los experimentos.

Elemento	Valor
Semilla	`SEED = 42` (`random` y `numpy`)
Mapa	12 × 12, costos {1, 3, 5}, 8 obstáculos
Hill Climbing	Vecindad swap, mejor mejora, 10 inicios aleatorios
Recocido Simulado	T_0 = 100, α = 0.9, T_min = 0.1, máximo 50 evaluaciones

Elemento	Valor
Algoritmo Genético	Población 10, 20 generaciones, torneo k = 2, cruza tipo OX1 simplificada, mutación swap p = 0.2, elitismo 1
Repeticiones	20 por metaheurística
Minimax / Alfa-Beta	Recursos [15, 10, 8, 5, 3, 2, 1], profundidad 5, MAX inicia
Medición de tiempo	`time.perf_counter()` en las Fases 2 a 4 (el prototipo usa `time.time()`)

5.3 Métricas y procedimiento

Se reporta el costo total de la ruta (menor es mejor), la media y la desviación estándar entre ejecuciones (desviación poblacional, `np.std`), el número de evaluaciones de la función de costo, y el tiempo de pared. Los datos por ejecución se exportan a `datos_experimentales.csv` (40 filas: 20 por algoritmo). Como verificación adicional realizada al preparar este informe, se enumeraron las 6 permutaciones posibles para confirmar el óptimo global. No se registraron las características del equipo de cómputo, por lo que los tiempos son comparables entre sí pero no entre equipos.

6. Resultados cuantitativos y visualizaciones

6.1 Búsqueda heurística (E1)

Tabla 6. Greedy BFS vs A* en el escenario definitivo.

Trayecto	Algoritmo	Costo	Nodos expandidos	Tiempo (s)	¿Óptimo?
S → D1	Greedy	11	12	0.000094	Sí
S → D1	A*	11	22	0.000096	Sí
D1 → D2	Greedy	22	16	0.000080	No
D1 → D2	A*	14	48	0.000201	Sí
D2 → D3	Greedy	15	12	0.000054	No
D2 → D3	A*	11	22	0.000093	Sí

En el escenario prototipo (trayecto I → E1, costos aleatorios entre 1 y 5), A* obtuvo costo 19 con 29 nodos expandidos y Greedy costo 20 con 11 nodos, el mismo patrón: Greedy expande menos nodos pero no garantiza la mejor ruta.

6.2 Matriz de costos y ruta óptima

Tabla 7. Costos mínimos entre puntos clave (calculados con A*; la matriz es simétrica y se omite la fila de F).

Desde \ Hasta	S	D1	D2	D3	F
S	0	11	13	16	22
D1	11	0	14	5	11

Desde \ Hasta	S	D1	D2	D3	F
D2	13	14	0	11	9
D3	16	5	11	0	6

Tabla 8. Costo de las 6 permutaciones posibles.

Orden de visita	Costo total	Nota
S → D1 → D3 → D2 → F	**36**	Óptimo global
S → D2 → D1 → D3 → F	38	Óptimo local de Hill Climbing (ver 6.3)
S → D2 → D3 → D1 → F	40	
S → D1 → D2 → D3 → F	42	
S → D3 → D1 → D2 → F	44	
S → D3 → D2 → D1 → F	52	Peor ruta

La ruta óptima suma 11 (S→D1) + 5 (D1→D3) + 11 (D3→D2) + 9 (D2→F) = **36**. La Figura 2 muestra el recorrido sobre el mapa; el trayecto evita las zonas de costo 5 y los obstáculos.

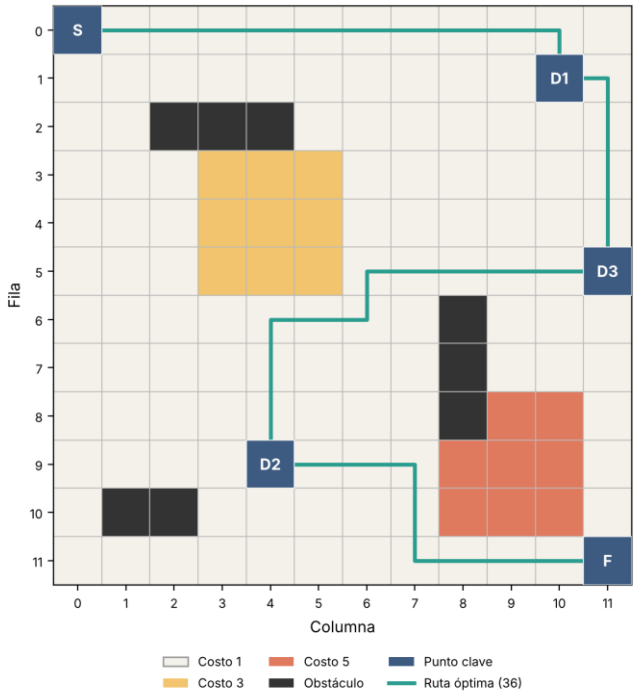


Figura 2. Mapa logístico y ruta óptima S → D1 → D3 → D2 → F (costo 36). Elaboración propia a partir de mapa_logistico.csv.

6.3 Hill Climbing (E2)

Las 10 ejecuciones partieron de soluciones con costo entre 36 y 44 y todas terminaron en **S → D1 → D3 → D2 → F con costo 36** (detalle en el Anexo B). Tres inicios (ejecuciones 2, 3 y 10) ya eran la solución óptima y en los demás casos el algoritmo descendió hasta ella.

Observación importante: la conclusión «siempre converge al mismo óptimo» describe esta muestra, no el algoritmo. Al revisar la vecindad swap, la permutación D2 → D1 → D3 (costo 38) es un **óptimo local verdadero**: sus tres vecinos cuestan 42, 44 y 40. Un inicio en esa permutación (probabilidad 1/6 por muestra) dejaría atascado al algoritmo; en 10 muestras la probabilidad de no tocarla es $(5/6)^{10} \approx 16\ \%$, lo que explica que no apareciera.

6.4 Recocido Simulado vs Algoritmo Genético (E3)

Tabla 9. Resultados estadísticos calculados a partir de datos_experimentales.csv.

Métrica (20 ejecuciones)	Recocido Simulado	Algoritmo Genético
Mejor costo	36	36
Costo promedio	36.00	36.10
Desviación estándar (poblacional)	0.0000	0.4359
Ejecuciones con costo óptimo (36)	20 de 20	19 de 20
Peor costo observado	36	38
Evaluaciones promedio	50	200
Tiempo promedio (s)	0.000111	0.001050
Desviación estándar del tiempo (s)	0.000012	0.000098

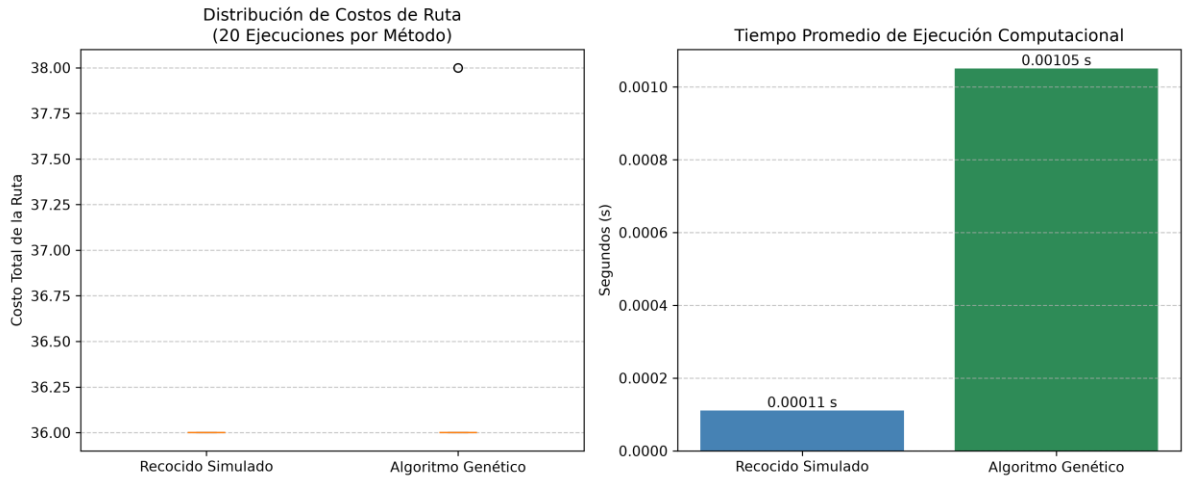


Figura 3. Izquierda: distribución del costo por método (20 ejecuciones). Derecha: tiempo promedio de ejecución.

En el diagrama de caja, el Recocido Simulado colapsa en una línea en 36; el Algoritmo Genético hace lo propio salvo por un valor atípico en 38. Aunque el AG usó 4 veces más evaluaciones, su tiempo promedio fue ≈ 9.5 veces mayor (≈ 5.3 µs por evaluación frente a ≈ 2.2 µs), lo que indica un costo adicional por gestión de población, selección y generación de permutaciones con `numpy`.

Nota sobre las cifras de tiempo: la consola del notebook mostró 0.000122 s y 0.001124 s en una ejecución, mientras que el CSV y la gráfica provienen de otra (0.000111 s y 0.001050 s). Los tiempos de microsegundos varían de una corrida a otra; las conclusiones se basan en el orden de magnitud y la razón entre métodos.

6.5 Teoría de juegos (E4)

Tabla 10. Matriz de pagos (utilidad J1, utilidad J2).

J1 \ J2	Bloquear Principal	Bloquear Secundaria	No Bloquear
Ruta Segura	(4, 1)	(4, 2)	**(5, 4)** ← Nash
Ruta Principal	(2, 5)	(6, 3)	(3, 2)
Ruta Secundaria	(3, 2)	(5, 4)	(4, 3)

- **Mejores respuestas de J1:** Bloquear Principal → Ruta Segura; Bloquear Secundaria → Ruta Principal; No Bloquear → Ruta Segura.
- **Mejores respuestas de J2:** Ruta Segura → No Bloquear; Ruta Principal → Bloquear Principal; Ruta Secundaria → Bloquear Secundaria.
- **Dominancia:** ninguno de los jugadores tiene una estrategia estrictamente dominante.
- **Equilibrio de Nash puro (único):** (Ruta Segura, No Bloquear) con pagos (5, 4). Es la única celda donde ambas mejores respuestas coinciden.

Interpretación: si el competidor no bloquea, el agente prefiere la ruta segura; y si el agente usa la ruta segura, bloquear no le da beneficio al competidor, por lo que ninguno tiene incentivo para desviarse unilateralmente.

6.6 Minimax vs Alfa-Beta (E5)

Tabla 11. Búsqueda adversaria con profundidad 5.

Métrica	Minimax	Alfa-Beta
Valor óptimo del juego	11	11
Nodos explorados	3,620	246
Podas realizadas	—	68
Reducción de nodos	—	93.20 %

Ambos algoritmos coinciden en el valor 11, lo cual valida la corrección de la poda. Un jugador voraz que tomara siempre el mayor recurso disponible produciría la misma diferencia (15 + 8 + 3) − (10 + 5) = 11 en este caso, lo que sugiere que el orden descendente de recursos y la profundidad limitada simplifican el juego.

6.7 Caso límite (E6)

Al pedir una ruta hacia (2, 2), que es un obstáculo, `es_valida` detecta la entrada inválida y, al forzar a A*, este agota la frontera y devuelve `None` sin lanzar excepciones. El resultado confirma el manejo controlado de un escenario sin solución. Cabe notar que las funciones devuelven costo 0 cuando no hay ruta, valor que podría confundirse con un costo real.

7. Análisis comparativo y discusión

7.1 A* frente a Greedy

Greedy redujo el número de nodos expandidos entre 45 % y 67 % en los tres trayectos, pero su ruta fue 57 % más costosa en D1→D2 (22 frente a 14) y 36 % más costosa en D2→D3 (15 frente a 11). A* pagó entre 1.8 y 3 veces más expansiones a cambio de optimalidad. Para decisiones logísticas donde el costo del recorrido importa más que unos microsegundos de cómputo, A* es la elección adecuada; Greedy solo sería preferible con restricciones de tiempo muy estrictas o mapas mucho mayores.

7.2 Hill Climbing, Recocido Simulado y Algoritmo Genético

- **Calidad:** los tres alcanzaron el óptimo (36). Esto ocurre porque el espacio tiene 6 soluciones y las 50 evaluaciones del Recocido Simulado superan con mucho ese número.
- **Estabilidad:** el Recocido Simulado fue perfecto en 20 ejecuciones; el Algoritmo Genético falló una vez. Como el único costo 38 corresponde a la permutación D2 → D1 → D3, el AG terminó en el óptimo local de la vecindad swap. Una explicación plausible, no verificada experimentalmente, es la pérdida de diversidad: con 3 genes, una cruza que hereda un solo gen y una población de 10, la población puede homogeneizarse antes de visitar la mejor permutación.
- **Eficiencia:** el Recocido Simulado fue ≈ 9.5 veces más rápido y usó 4 veces menos evaluaciones.
- **Significancia:** 19/20 frente a 20/20 no basta para afirmar que el AG sea peor en calidad; es un indicio de menor estabilidad bajo estos parámetros.

Estos resultados **no** deben extrapolarse a instancias grandes. Con 10 o más entregas (más de 3.6 millones de permutaciones) la enumeración deja de ser viable y las diferencias entre las metaheurísticas serían más informativas. Además, los parámetros del Recocido ($T_0 = 100$ frente a Δ típicos de 2 a 8) hacen que al inicio casi todo movimiento se acepte ($\exp(-4/100) \approx 0.96$), comportándose como una caminata aleatoria que luego se enfría; en un espacio tan pequeño eso es suficiente, pero no está calibrado.

7.3 Decisión estratégica

El equilibrio de Nash identifica la combinación estable bajo la suposición de que ambos jugadores razonan racionalmente y de forma simultánea. Su valor práctico depende de que los pagos reflejen la realidad; como fueron asignados por el diseñador, el resultado ilustra el método más que una predicción logística. Por otro lado, la búsqueda adversaria modela una interacción secuencial; ahí Alfa-Beta demuestra su utilidad: misma decisión con ≈ 7 % de los nodos.

8. Pruebas, limitaciones y amenazas a la validez

8.1 Pruebas realizadas

- **Equivalencia Minimax / Alfa-Beta:** el código compara los valores y ambos devuelven 11.
- **Optimalidad de A*:** el costo de la mejor permutación (36) coincide con la enumeración exhaustiva de las 6 rutas.
- **Validez de permutaciones:** la cruza y la mutación del AG siempre producen permutaciones sin repetidos ni faltantes.
- **Entradas inválidas:** el caso (2, 2) se detecta y no genera excepciones.
- **Reproducibilidad:** semilla fija y escenario en CSV; la interfaz reejecuta el notebook completo.

No existen pruebas unitarias automatizadas; las verificaciones anteriores son comprobaciones dentro del propio notebook.

8.2 Limitaciones y amenazas a la validez

Tabla 12. Limitaciones y amenazas a la validez.

Tipo	Descripción
Validez de constructo	Los pagos del juego 3 × 3 y la utilidad del juego de recursos son asignaciones arbitrarias; no se derivan del mapa ni de datos reales. El juego de recursos se evalúa solo a profundidad 5 de 7.
Validez interna	Los parámetros del Recocido y del AG no fueron calibrados ni sometidos a análisis de sensibilidad. La cruza OX1 es una versión simplificada de un solo gen. El tiempo de los algoritmos se mide en microsegundos, comparable al ruido del sistema.
Validez de conclusión	Con 20 repeticiones y un único valor atípico no se aplicó ninguna prueba estadística. Hill Climbing se probó con solo 10 inicios, insuficientes para descartar óptimos locales.
Validez externa	Un solo mapa de 12 × 12, tres entregas y un solo vehículo. Con 6 soluciones posibles todos los métodos son «óptimos»; no se demuestra escalabilidad.
Diseño de software	Las fases comparten estado global y dependen del orden de ejecución. La primera celda mantiene un escenario prototipo con nombres que luego se sobrescriben. `a_star` y `greedy_bfs` devuelven costo 0 si no hay ruta.
Entorno de cómputo	No se registraron versión de Python/librerías ni hardware. Los tiempos varían entre ejecuciones (p. ej., consola vs. CSV).

9. Conclusiones y trabajo futuro

9.1 Conclusiones

1. El sistema resuelve el ruteo de tres entregas con costo mínimo **36** (S → D1 → D3 → D2 → F), verificado por enumeración exhaustiva.

2. A* con distancia de Manhattan garantiza optimalidad, mientras que Greedy sacrifica calidad (hasta 57 % más de costo en un trayecto) por menos expansiones.
3. Precalcular una matriz de costos con A* hace que evaluar una ruta sea trivial y permite probar con facilidad distintos optimizadores combinatorios.
4. El Recocido Simulado fue más rápido y estable que el Algoritmo Genético en este problema, pero la diferencia refleja eficiencia y no capacidad de resolver problemas difíciles, por el tamaño reducido del espacio.
5. El análisis de teoría de juegos identifica un único equilibrio de Nash puro (Ruta Segura, No Bloquear), y Alfa-Beta reproduce el resultado de Minimax con 93.20 % menos nodos.
6. La interfaz en Streamlit facilita la presentación reproducible de resultados, gráficas y datos.

9.2 Trabajo futuro

- Aumentar la escala: más entregas (10 a 20), mapas aleatorios múltiples y comparación contra un óptimo exacto (programación dinámica de Held-Karp) o contra 2-opt/Or-opt.
- Calibrar parámetros (T_0, α, tamaño de población, operadores) con diseños factoriales y reportar intervalos de confianza y pruebas no paramétricas (p. ej., Mann-Whitney).
- Añadir restricciones realistas: capacidad, ventanas de tiempo, tráfico dinámico y flotas múltiples (VRP).
- Derivar los pagos de la matriz del juego a partir de simulaciones sobre el mapa y explorar equilibrios mixtos.
- Mejorar el juego adversario: profundidad completa, tabla de transposición, ordenamiento de movimientos y una función de evaluación más informativa.
- Modularizar el código en un paquete con pruebas unitarias (`pytest`), registrar versiones y hardware, y devolver `None`/`inf` de forma consistente cuando no hay ruta.

10. Referencias y anexos técnicos

10.1 Referencias

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

Davis, L. (1985). Applying adaptive algorithms to epistatic domains. *Proceedings of the 9th IJCAI*, 162–164.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680.

Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence, 6*(4), 293–326.

Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equation of state calculations by fast computing machines. *The Journal of Chemical Physics, 21*(6), 1087–1092.

Nash, J. F. (1950). Equilibrium points in n-person games. *Proceedings of the National Academy of Sciences, 36*(1), 48–49.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson.

Anexo A. Archivos y reproducibilidad

Tabla A1. Archivos del proyecto.

Archivo	Contenido
`proyecto2.ipynb`	Notebook con las fases 1 a 8.
`app.py`	Interfaz Streamlit: `streamlit run app.py` (se abre en http://localhost:8501).
`mapa_logistico.csv`	Mapa 12 × 12 del escenario definitivo (S, F, D1–D3, X y costos).
`escenario.csv`	Escenario del prototipo inicial (I, F, E1–E3).
`datos_experimentales.csv`	40 filas: costo, evaluaciones, tiempo y algoritmo de cada ejecución.
`graficas_resultados.png`	Boxplot de costos y barras de tiempo promedio.

Dependencias: `numpy`, `pandas`, `matplotlib`, `streamlit`. Semilla: 42. Para reproducir los resultados, ejecutar el notebook de principio a fin; la interfaz lo hace automáticamente.

Anexo B. Detalle de Hill Climbing (10 ejecuciones)

Tabla B1. Resultados de Hill Climbing por ejecución.

#	Solución inicial	Costo inicial	Óptimo local alcanzado	Costo final
1	D2, D3, D1	40	D1, D3, D2	36
2	D1, D3, D2	36	D1, D3, D2	36
3	D1, D3, D2	36	D1, D3, D2	36
4	D2, D3, D1	40	D1, D3, D2	36
5	D3, D1, D2	44	D1, D3, D2	36
6	D1, D2, D3	42	D1, D3, D2	36
7	D1, D2, D3	42	D1, D3, D2	36
8	D1, D2, D3	42	D1, D3, D2	36
9	D2, D3, D1	40	D1, D3, D2	36
10	D1, D3, D2	36	D1, D3, D2	36

Anexo C. Preguntas guía para la defensa individual

Tabla C1. Preguntas y respuestas de preparación.

Pregunta	Respuesta breve
¿Por qué la distancia de Manhattan es admisible aquí?	Porque cada paso cuesta al menos 1 y solo hay movimientos ortogonales, así que h nunca supera el costo real restante. También es consistente.
¿Por qué Greedy puede dar peor ruta?	Ignora g(n): sigue el nodo que parece más cercano sin considerar el costo acumulado (p. ej., zonas de costo 3 o 5).

Pregunta	Respuesta breve
¿Cuál es la complejidad de evaluar una ruta?	O(n) con la matriz precalculada (4 sumas para n = 3), en lugar de ejecutar A* en cada evaluación.
¿Hay óptimos locales en Hill Climbing?	Sí: D2 → D1 → D3 (costo 38) con vecindad swap. Las 10 muestras no cayeron ahí por azar.
¿Por qué falló una vez el AG?	Terminó en la permutación de costo 38; hipótesis: pérdida de diversidad con población 10, 3 genes y cruza de un solo gen.
¿Qué garantiza Alfa-Beta?	Mismo valor que Minimax; solo descarta ramas que no pueden influir en la decisión en la raíz (α ≥ β).
¿Por qué el Nash es (Segura, No Bloquear)?	Es la única celda donde ambas mejores respuestas coinciden: ni J1 ni J2 mejoran desviándose solos.
¿Qué limita las conclusiones?	Espacio de 6 soluciones, mapa único, pagos subjetivos, parámetros sin calibrar y tiempos en microsegundos.